

TIGER ANALYTICS APP ENGINEERING ASSIGNMENT – Non-Functional Requirements

1. Performance

- Elasticsearch provides sub-second full-text and filtered search without hitting the database
- Bulk indexing batches records together instead of individual writes
- Paginated responses prevent large data transfers over the network

2. Scalability

- Elasticsearch sharding distributes data across nodes as volume grows
- MySQL unique constraints and connection pooling (HikariCP) handle concurrent uploads

3. Availability

- Dual-store architecture means if Elasticsearch goes down, MySQL still holds all data
- Elasticsearch can be fully rebuilt from MySQL at any time.
- Spring Boot Actuator health endpoints enable readiness/liveness probes for container orchestration

4. Data Integrity

- Unique constraint on (store_id, sku, date) prevents duplicate pricing records at the database level
- Duplicate detection logic during upload of csv files updates existing records instead of creating duplicates
- Row-level CSV validation rejects bad data and returns detailed error messages per row

5. Security

- CORS configuration restricts API access to known frontend origins only
- Input validation at CSV parsing layer prevents malformed data from entering the system
- UUID primary keys are non-sequential, making record enumeration attacks impractical

6. Internationalization

- Elasticsearch asciifolding filter enables accent-insensitive search (e.g., "Cafe" matches "Cafe Latte Powder")

TIGER ANALYTICS APP ENGINEERING ASSIGNMENT – Non-Functional Requirements

- UTF-8 encoding throughout the stack (Java, MySQL, ES) supports multi-language product names
- Angular currency and date pipes format values based on user locale

7. Maintainability

- Clear separation of concerns: JPA repositories for writes, ES repositories for reads, dedicated service layer
- Separate repository packages (repository.jpa vs repository.elasticsearch) avoid Spring Data module conflicts
- CSV parsing, indexing, and business logic are in distinct service classes

8. Data Consistency

- MySQL ensures reliable writes with ACID transactions
- Elasticsearch is treated as a read-optimized projection that can be rebuilt anytime
- Upload flow writes to MySQL first, then indexes to ES — if ES fails, data is still safe in MySQL

9. Usability

- Drag-and-drop CSV upload with file type validation before sending to server
- Search with multiple filter criteria (store, SKU, product name, date range, price range) in a single query
- Inline edit and delete with confirmation dialogs prevent accidental data modifications

10. Observability

- @Slf4j logging on service methods tracks upload counts, errors, and processing results
- Spring Boot Actuator exposes health, metrics, and info endpoints out of the box
- CSV upload response includes detailed error breakdown (total, success, failed, per-row errors)